vmsOpenAcars — Documentación de Arquitectura

> Versión del documento: 0.3.3
> Última actualización: 2026-04-26

¿Qué es vmsOpenAcars?

Cliente ACARS de escritorio para Windows que conecta un simulador de vuelo con una aerolínea virtual basada en **phpVMS v7**. El piloto vuela en el simulador y la aplicación registra, valida y envía el PIREP automáticamente, sin intervención manual.

Stack Técnico

Componente	Tecnología
Plataforma	.NET Framework 4.8, C# 7.3
UI	Windows Forms (WinForms)
Sim → App	FSUIPC / FSUIPC7 — FSUIPCClientDLL 3.3.16
App → VA	phpVMS v7 REST API
Serialización	Newtonsoft.Json 13
Plan de vuelo	SimBrief API (JSON)
PDF	PdfiumViewer 2.13 + pdfium.dll x64
Clima / QNH	METAR vía WeatherService
Localización	JSON (en.json / es.json)
Configuración	App.config + Settings.settings

Estructura de Carpetas

```
vmsOpenAcars/
├── Controls/              Controles visuales personalizados (gauges, engine
monitor)
├── Core/
│   ├── Flight/            FlightManager, FlightTimer
│   └── Helpers/           AppInfo
├── docs/                  Esta documentación
├── Helpers/               AppConfig, Constants, FlightPhaseHelper, L
(localización), UnitConverter
├── Languages/             en.json, es.json
├── Models/                Entidades de dominio (SimbriefPlan, Flight, Pirep,
FlightPhase…)
```

```
├── Properties/             AssemblyInfo, Resources, Settings
├── Services/               Servicios de infraestructura y negocio
├── UI/
│   ├── Forms/              MainForm, FlightPlannerForm, OFPViewerForm,
SettingsForm…
│   └── Theme.cs            Paleta de colores centralizada
└── ViewModels/             MainViewModel
```

Arquitectura General

El proyecto sigue un patrón **MVVM ligero**:

```
MainForm.cs  (Vista — WinForms)
    │
    └── MainViewModel.cs  (ViewModel — coordinación, eventos, lógica de botones)
            │
            ├── FlightManager.cs          Máquina de estados del vuelo
            ├── FsuipcService.cs          Polling del simulador (FSUIPC)
            ├── ApiService.cs             HTTP client → phpVMS REST API
            ├── PhpVmsFlightService.cs    Vuelos, bids, PIREPs
            ├── SimbriefEnhancedService.cs Plan de vuelo + OFP PDF
            └── WeatherService.cs         QNH vía METAR
```

La comunicación ViewModel → Vista se hace mediante **eventos** (`Action<T>`, `Func<T>`). La Vista nunca llama directamente a servicios de infraestructura.

Fases de Vuelo

La máquina de estados central (`FlightManager`) gestiona las siguientes fases:

```
Idle
 └► Boarding
      └► Pushback ──────────────────────────────────────┐
      └► TaxiOut (directo sin pushback)                  │
           └► TakeoffRoll                                │ (pushback → taxi)
                └► Takeoff                               │
                     └► Climb                            │
                          └► Enroute                     │
                               └► Descent                │
                                    └► Approach          │
                                         │               │
                                         ├► Climb (go-around)
                                         │
                                         └► (touchdown)
                                              └► AfterLanding
                                                   └► TaxiIn
                                                        └► OnBlock
```

```
                                                                    └─► Arrived
                                                                        └─► Completed
```

Cada transición envía un código de estado al servidor phpVMS:

Fase	Código phpVMS
Boarding	`BST`
Pushback	`PBT`
TaxiOut / TaxiIn	`TXI`
Takeoff	`TOF`
Climb	`ICL`
Enroute	`ENR`
Descent	`APR`
Approach	`FIN`
Landing	`LDG`
OnBlock / Completed	`ARR`

Módulos

FsuipcService

Servicio de lectura del simulador. Opera en un timer a **50 ms** e interpola los offsets FSUIPC.

Offsets principales leídos:

Dato	Offset	Conversión
Latitud	0x0560	`raw × 90 / (10001750 × 65536²)`
Longitud	0x0568	`raw × 360 / 2⁶⁴`
Altitud MSL	0x0570	`(raw / 65536²) × 3.28084` ft
Heading	0x0580	`raw × 360 / 2³²`
Ground Speed	0x02B4	`(raw / 65536) × 1.94384` kts
Vertical Speed	0x02C8	`(raw / 256) × 196.85` fpm
IAS	0x02BC	`raw / 128` kts
Radar Altímetro	0x31E4	metros → feet
Luces	0x0D0C	bitfield (ver tabla de luces)
QNH (Kohlsman)	0x0330	`raw / 16.0` hPa

Dato	Offset	Conversión
Viento velocidad	0x0E90	kts directo
Viento dirección	0x0E92	`raw × 360 / 65536` grados
OAT	0x0E8C	`raw / 256` °C
G-Force	0x11BA	`raw / 625.0`

Bitfield de luces (0x0D0C):

Bit	Luz
0	NAV
1	BEACON
2	LANDING
3	TAXI
4	STROBE
5	SEAT BELT SIGN

Detección de eventos con debounce:

Evento	Debounce
NAV light	1.5 s
STROBE light	1.5 s
BEACON light	1.5 s
LANDING light	1.5 s
TAXI light	1.5 s
Parking Brake	2.0 s
Flaps	500 ms + histéresis 1%
Touchdown / Liftoff	2.0 s + umbral GS

Telemetría adaptativa por fase:

Fase	Intervalo
Taxi	30 s
Takeoff / Approach	5 s
Climb / Descent	15 s
Enroute	30 s

Fase	Intervalo
Otros	30 s

FlightManager

Núcleo de la lógica del vuelo. Recibe `RawTelemetryData` cada ciclo y:

1. Actualiza propiedades públicas (altitud, GS, VS, luces, motores…)
2. Aplica **debounce de luces** (2 s) antes de actualizar los campos internos usados en compliance
3. Detecta transiciones de fase con umbrales relativos al plan de vuelo
4. Llama a `CheckProcedureAtPhaseEntry()` en cada transición
5. Llama a `CheckViolations()` cada ciclo mientras el avión está airborne y hay PIREP activo

Cálculo de AGL

Condición	Fuente
En tierra (`IsOnGround = true`)	`0` siempre
Fase **Enroute**	Radar altímetro (`RadarAltitude`); fallback a baro si = 0
Resto de fases aéreas	`CurrentAltitude - ReferenceAirportElevation`

`ReferenceAirportElevation` devuelve la elevación del aeropuerto de **salida** en fases de despegue/climb, y la elevación del aeropuerto de **destino** en fases de llegada.

> El radar altímetro **no se usa en aproximación** porque en terrenos montañosos (Andes, Alpes, Himalaya) lee la orografía bajo las ruedas, que puede estar muy por encima del aeropuerto, dando un AGL incorrecto.

Detección de Go-Around

Un go-around se confirma si se cumplen **todos** los criterios simultáneamente:

Criterio	Valor
VS positivo	> 600 fpm
Duración sostenida	≥ 8 segundos consecutivos
Tiempo mínimo en fase Approach	≥ 30 segundos
AGL (MSL − destElev)	entre 100 y 3 000 ft

El umbral de 600 fpm elimina falsos positivos por turbulencia o ajustes de pitch en final.

Compliance de Procedimientos (penalizaciones)

Momento	Verificación	Penalización
Motores ON	BEACON encendida	−5 pts

Momento	Verificación	Penalización
Pushback	NAV encendida	−5 pts
TaxiOut	NAV + TAXI encendidas	−5 pts c/u
TakeoffRoll	STROBE + LANDING encendidas	−5 pts c/u
Vuelo < 10 000 ft	LANDING encendida	−5 pts
Despegue	QNH ±2 hPa vs METAR origen	−5 pts
Approach	QNH ±2 hPa vs METAR destino	−5 pts

ScoringService

Calcula un score de 0–100 al finalizar el vuelo. El score comienza en 100 y se aplican deducciones:

Criterio	Máx. deducción	Escala
Landing rate	−40 pts	≤100 fpm: 0 / ≤200: −5 / ≤300: −15 / ≤400: −25 / ≤600: −35 / >600: −40
G-Force touchdown	−15 pts	≤1.3g: 0 / ≤1.5g: −7 / >1.5g: −15
Bank angle touchdown	−10 pts	≤2°: 0 / ≤5°: −5 / >5°: −10
Pitch angle touchdown	−10 pts	1°–5°: 0 (ideal) / plano o nariz abajo: −5 / <−2°: −10
Overspeed (vs Vmo)	−15 pts	0 eventos: 0 / 1: −7 / ≥2: −15
Luces/procedimientos	−10 pts	2 pts × violación, máx −10

Landing ratings: Butter (≤100 fpm) · Smooth · Normal · Hard · Very Hard · Slam (≥600 fpm)

AircraftPerformanceTable y Detección de Overspeed

¿Qué es Vmo?

Vmo (Velocity Maximum Operating) es la velocidad máxima operativa publicada en el FCOM/AFM de cada aeronave, expresada en nudos IAS (Indicated Airspeed). Volar por encima de ella puede causar daño estructural, activar advertencias de overspeeed en el cockpit y, en operación real, resulta en un informe de seguridad obligatorio.

En vmsOpenAcars, el Vmo es el único umbral de velocidad usado para penalizar al piloto. No se implementa Mmo (límite de Mach) porque FSUIPC expone IAS directamente y la conversión IAS↔Mach requiere datos de temperatura y presión que varían con la altitud.

Resolución del Tipo de Aeronave

Cuando se inicia un vuelo, `FlightManager` consulta `AircraftPerformanceTable.Get(icaoType)` con el código ICAO del tipo procedente del plan SimBrief o del PIREP. La resolución sigue este orden de prioridad:

```
1. Match exacto (case-insensitive)   "B738"  → Boeing 737-800, Vmo 340 kts
        ↓ no encontrado
2. Prefijo de 4 chars                "B38M"  → prefijo "B38" no existe
        ↓ no encontrado
3. Prefijo de 3 chars                "B38M"  → prefijo "B3" → familia B737 MAX,
Vmo 340 kts
        ↓ no encontrado
4. Default genérico                  320 kts, categoría "Generic (unknown type)"
```

Esta cascada evita que un código de aeronave desconocido (variante de livery, tipo personalizado de la VA) cause una penalización incorrecta por overspeed.

Tabla Completa de Vmo por Categoría

Pistones ligeros y GA

Tipo ICAO	Aeronave	Vmo (kts)
C172	Cessna 172	163
C182	Cessna 182	175
C208	Cessna Caravan	175
PA28	Piper PA-28	148
PA44	Piper Seminole	169
BE58	Beechcraft Baron 58	195
BE20	King Air 200	260
BE30	King Air 300/350	260
PC12	Pilatus PC-12	210

Turboprops regionales

Tipo ICAO	Aeronave	Vmo (kts)
AT42–AT46	ATR 42 (todas variantes)	250
AT72–AT76	ATR 72 (todas variantes)	250
DH8A / DH8B	Dash 8-100/200	220
DH8C / DH8D	Dash 8-300/400 (Q400)	260
SB20	Saab 2000	290
SF34	Saab 340	250

Tipo ICAO	Aeronave	Vmo (kts)
E120	Embraer 120	255

Jets regionales

Tipo ICAO	Aeronave	Vmo (kts)
CRJ2	CRJ-200	320
CRJ7	CRJ-700	320
CRJ9	CRJ-900	320
CRJX	CRJ-1000	320
E135 / E145	ERJ-135 / ERJ-145	320
E170 / E175	Embraer E170 / E175	320
E190 / E195	Embraer E190 / E195	320

Narrow-body jets

Tipo ICAO	Aeronave	Vmo (kts)
A318–A321	Airbus A318/319/320/321	350
A20N / A21N	A320neo / A321neo	350
B731–B739	Boeing 737 Clásico y NG	340
B37M–B3XM	Boeing 737 MAX 7/8/9/10	340
MD82 / MD83	McDonnell Douglas MD-80	340

Wide-body jets

Tipo ICAO	Aeronave	Vmo (kts)
A332 / A333 / A339	Airbus A330-200/300/900neo	330
A342–A346	Airbus A340 (todas variantes)	330
A359 / A35K	Airbus A350-900/1000	330
A388	Airbus A380-800	330
B752 / B753	Boeing 757-200/300	350
B762–B764	Boeing 767-200/300/400	350
B772–B77W	Boeing 777-200/300/ER/LR	330
B779	Boeing 777X	330
B787–B78X	Boeing 787-8/9/10	330

Tipo ICAO	Aeronave	Vmo (kts)
B744 / B748	Boeing 747-400/8	365
B74F / B74S	Boeing 747-400F / 747SP	365

Prefijos fallback (3 chars)

Si el tipo exacto no está en la tabla, se busca el prefijo de 3 caracteres:

Prefijo	Familia	Vmo (kts)
A32	A320 family	350
A33	A330 family	330
A34	A340 family	330
A35	A350 family	330
A38	A380 family	330
B73	B737 family	340
B3	B737 MAX	340
B74	B747 family	365
B75	B757 family	350
B76	B767 family	350
B77	B777 family	330
B78	B787 family	330
AT4	ATR 42 family	250
AT7	ATR 72 family	250
DH8	Dash 8 family	260
DHC	DHC family	215
CRJ	CRJ family	320
E17	E-jet 170	320
E19	E-jet 190	320
BE2	King Air	260
BE3	King Air 350	260
C20	Caravan	175

Si ningún prefijo coincide → **320 kts** (default genérico conservador).

Ciclo de Detección de Overspeed

`CheckViolations()` se llama **una vez por ciclo de telemetría** (cada ~50 ms) mientras el avión está airborne y hay un PIREP activo:

```
Por cada ciclo:
    IAS actual > Vmo?
        SÍ y no estaba en overspeed antes → _overspeedCount++ + log
        NO                                → reset flag _wasOverspeed
```

El flag `_wasOverspeed` actúa como latch: un evento de overspeed sostenido (ej. 30 segundos a 360 kts) cuenta como **un solo evento**, no como 600 eventos (uno por ciclo). El contador solo incrementa en la **transición** normal→overspeed.

Deducción de Score por Overspeed

Eventos registrados	Deducción
0	0 pts
1	−7 pts
≥ 2	−15 pts (máximo)

La penalización máxima es −15 pts sobre 100. Incluso con múltiples episodios de overspeed el score no puede caer más de 15 pts solo por este criterio; las penalizaciones de luces y aterrizaje acumulan el resto.

Ejemplo Práctico

Un piloto vuela un A320 (tipo `A320`, Vmo = **350 kts**) y durante el descenso mantiene 355 kts IAS durante 45 segundos, luego reduce:

```
Ciclo  1: IAS 355 > 350 → _wasOverspeed=false → _overspeedCount=1, log "⚠
OVERSPEED: 355 kts"
Ciclos 2–90: IAS 355 > 350 → _wasOverspeed=true → sin cambio (mismo evento)
Ciclo 91: IAS 348 < 350 → _wasOverspeed=false
```

Resultado: 1 evento → **−7 pts** en el score final.

SimbriefEnhancedService

`GenerateDispatchUrl()` — Genera URL para pre-cargar el plan en SimBrief:

- Hora de salida: UTC actual + 30 min
- Parámetros: aerolínea, número, origen, destino, tipo aeronave, matrícula, ruta, CI, unidades

`FetchAndParseOFP()` — Descarga y parsea el JSON de la API de SimBrief:

- Construye `SimbriefPlan` completo: routing, combustible, pesos, tiempos, elevaciones, URL del PDF

- Maneja que `files.pdf` puede ser string o objeto `{name, link}`

OFP PDF — Flujo Completo

```
FlightPlannerForm (acepta OFP)
    │
    ├─► SetActivePlan(plan)
    └─► DownloadOFPPdfAsync()  ← background, fire-and-forget
            │
            └─► plan.LocalPdfPath = "/temp/vmsOFP_xxxx.pdf"

BtnOfp_Click
    ├─ GetCachedOFPPath()  ←─ archivo en disco?
    │       ├─ Sí → OFPViewerForm(cachedPath)  [instantáneo]
    │       └─ No → DownloadOFPPdfAsync() → OFPViewerForm(newPath)
    │
    └── OFPViewerForm.OnFormClosed → File.Delete(tempPath)
```

WeatherService

Obtiene QNH desde METAR para un aeropuerto ICAO dado. Usado en `CheckQnhAsync()` al entrar en TakeoffRoll y en Approach.

Modelos Principales

SimbriefPlan

```
// Vuelo
string FlightNumber, Airline, Origin, Destination, Alternate, Route
int DestinationElevation, OriginElevation, CruiseAltitude, EstTimeEnroute
long TimeGenerated, ScheduledOffTime

// Aeronave
string Aircraft, AircraftIcao, Registration, FlightId, BidId

// Combustible / Pesos
double BlockFuel, DepartureFuel, PayLoad, ZeroFuelWeight
string Units   // "KG" | "LBS"
int PaxCount

// PDF
string PdfUrl       // URL SimBrief
string LocalPdfPath // Caché local (null si no descargado)
```

FlightPhase (enum)

`Idle → Boarding → Pushback → TaxiOut → TakeoffRoll → Takeoff → Climb → Enroute → Descent → Approach → Landing → Landed → AfterLanding → TaxiIn → OnBlock → Arrived → Completed`

Localización

Dos archivos JSON en `Languages/`: `en.json` y `es.json`.
Acceso mediante el helper estático `L._("key")` importado con `using static vmsOpenAcars.Helpers.L`.
El idioma se selecciona en `SettingsForm` y se persiste en `App.config`.

Configuración (App.config / AppConfig)

Clave	Default	Descripción
`polling_interval_ms`	50	Intervalo de polling FSUIPC
`update_interval_taxi`	30	Telemetría en taxi (s)
`update_interval_takeoff`	5	Telemetría en despegue (s)
`update_interval_climb`	15	Telemetría en subida (s)
`update_interval_cruise`	30	Telemetría en crucero (s)
`update_interval_descent`	15	Telemetría en descenso (s)
`update_interval_approach`	5	Telemetría en approach (s)
`fuel_tolerance_percent`	10	Tolerancia combustible (%)
`fuel_tolerance_absolute`	50	Tolerancia combustible (kg)
`simbrief_civalue`	30	Cost Index para SimBrief
`simbrief_units`	lbs	Unidades combustible SimBrief

Requisitos de Instalación

1. **Simulador**: MSFS 2020/2024, Prepar3D v4/v5, o X-Plane 11/12
2. **FSUIPC**: FSUIPC4, FSUIPC6 o FSUIPC7 instalado y activo
3. **pdfium.dll** (x64): incluido en el build (`PdfiumViewer.Native.x86_64.no_v8-no_xfa`)
4. **Plataforma destino**: x64 (`Prefer32Bit=false` en el proyecto)
5. **.NET Framework 4.8** en el sistema

Notas de Build

- Configuración **Release**: copia `App.Release.config` sobre `vmsOpenAcars.exe.config` en post-build
- `pdfium.dll` se copia siempre al directorio de salida (`CopyToOutputDirectory=Always`)
- `Languages/*.json` se copian con `PreserveNewest`